

Informe del TP Final

Orden.ts

```
export type EstadoOrden = 'PENDIENTE' | 'COMPLETADA';
```

Definimos `EstadoOrden` como un **type literal** en vez de un enum. En TypeScript, los type literals son más livianos que los enums (no generan código JavaScript extra) y son suficientes cuando los valores son fijos y no necesitan lógica asociada. El nombre del tipo es `EstadoOrden` y no simplemente `Estado` porque en un sistema con múltiples entidades con estado (Camion, Robot, Orden), los nombres ambiguos generan confusión.

```
export class Orden {  
  public estado: EstadoOrden = 'PENDIENTE';  
  public paquete: Paquete | null = null;  
  
  constructor(  
    public readonly id: string,  
    public readonly camionId: string,  
    public readonly paqueteId: string,  
    public readonly tipoPaquete: string,  
    public readonly peso: number,  
    public readonly vencimiento: Date | null  
  ) {}  
}
```

Los campos del constructor son todos `readonly` porque representan la identidad de la orden — son datos que vienen del CSV y no cambian nunca. Una orden es siempre para el mismo camión, el mismo paquete, el mismo tipo. Si esos valores pudieran cambiar, sería una orden distinta.

`estado` y `paquete` en cambio no son `readonly` porque sí cambian durante el ciclo de vida: `estado` pasa de PENDIENTE a COMPLETADA, y `paquete` arranca en `null` y se asigna cuando el robot lo instancia físicamente.

`paquete` es `Paquete | null` y no directamente un `Paquete` porque la orden existe antes de que el paquete sea creado en el sistema. En RECEPCION, el paquete no existe hasta que el robot lo retira del camión.

```
public completar(): void {
    this.estado = 'COMPLETADA';
}

public estaCompleta(): boolean {
    return this.estado === 'COMPLETADA';
}
}
```

Solo agregamos dos métodos porque son las únicas dos operaciones que puede sufrir una orden desde que se crea: completarse, y ser consultada sobre si está completa. No hay `cancelar()`, no hay `reabrir()`, porque el sistema no los necesita — agregar métodos que no se usan viola YAGNI y ensucia la interfaz pública de la clase.

Ambos son `public` porque se llaman desde fuera de la clase: `completar()` lo llama `Robot.realizarOperacion()` y `estaCompleta()` lo llama `Camion.todasCompletadas()` y `ControladorAlmacen.asignarOrdenes()`.

SOLID:

- **SRP:** Orden solo sabe de sí misma. No sabe de robots, camiones, ni del almacén.
- **ISP:** La interfaz pública es mínima — solo lo necesario. Nada que no se use.

GRASP:

- **Information Expert:** Orden es la experta en su propio estado de completitud. `estaCompleta()` vive acá y no en el Controlador, porque la información (`estado`) está acá.
- **Low Coupling:** No importa ni referencia ninguna otra entidad excepto `Paquete`, que es su relación natural de dominio.

Paquete.ts

```
export abstract class Paquete {
  constructor(
    public readonly id: string,
    public readonly peso: number,
    public readonly categoria: string,
    public posicion: Celda | null = null
  ) {}
}
```

`Paquete` es **abstracta** porque en el dominio nunca existe "un paquete genérico sin tipo" — siempre es comestible o general. Hacerla abstracta impide instanciarla directamente, lo cual sería un error de modelado.

`id`, `peso` y `categoria` son `readonly` porque son la identidad del paquete. `posicion` en cambio no es `readonly` porque el paquete se mueve: arranca en `null`, pasa por el muelle, llega a la estantería. Rastrear la posición en el paquete mismo sigue el principio de **Information Expert** — el paquete sabe dónde está.

`posicion` es `Celda | null` porque el paquete existe antes de ser ubicado físicamente (cuando se crea la Orden pero antes de que el robot lo tome del camión).

```
export class PaqueteComestible extends Paquete {
  constructor(
    id: string, peso: number,
    public readonly vencimiento: Date,
    posicion: Celda | null = null
  ) {
    super(id, peso, 'COMESTIBLE', posicion);
  }
}

export class PaqueteGeneral extends Paquete {
  constructor(id: string, peso: number, posicion: Celda | null = null) {
    super(id, peso, 'GENERAL', posicion);
  }
}
```

`PaqueteComestible` agrega `vencimiento: Date` — el único campo que lo diferencia estructuralmente. Es `readonly` porque la fecha de vencimiento no cambia. `vencimiento` no es opcional (`Date | null`) porque si es comestible, siempre tiene fecha — si no tuviera, sería un error de datos del CSV.

`PaqueteGeneral` no agrega campos porque no los necesita. Su diferencia es semántica: tiene distinta prioridad en el ordenamiento FEFO.

El campo `categoria` en la base se pasa como literal desde cada subclase (`'COMESTIBLE'`, `'GENERAL'`). Esto permite identificar el tipo sin usar `instanceof` en todos los lugares — por ejemplo, en `obtenerEstado()` se usa `p.categoria` directamente para serializar al frontend.

SOLID:

- **SRP:** Cada clase tiene una sola razón de cambio. Si cambia la lógica de vencimiento, solo toca `PaqueteComestible`.
- **OCP:** Agregar `PaqueteFragil` o `PaqueteRefrigerado` no requiere modificar `Paquete` ni las subclases existentes.
- **LSP:** Cualquier lugar que espera un `Paquete` funciona igual con `PaqueteComestible` o `PaqueteGeneral` — no hay comportamiento que rompa al sustituir.

GRASP:

- **Polymorphism:** El comportamiento diferente entre tipos (prioridad de ordenamiento) se modela con subclases, no con `if (tipo === 'COMESTIBLE')`.
- **Information Expert:** Cada subclase sabe sus propios datos. `PaqueteComestible` sabe su vencimiento; `Paquete` sabe su posición.

Celda.ts

```
export type TipoCelda = 'PASILLO' | 'ESTANTERIA' | 'MUELLE'
| 'BASE_CARGA';

export abstract class Celda {
  public ocupada: boolean = false;
  public reservada: any | null = null;
  constructor(
```

```

    public readonly x: number,
    public readonly y: number,
    public readonly tipo: TipoCelda
  ) {}
}

```

`Celda` es abstracta por la misma razón que `Paquete` — en el dominio no existe "una celda sin tipo". Toda celda es pasillo, estantería, muelle o base de carga.

`x`, `y` y `tipo` son `readonly` porque la posición y el tipo de una celda no cambian nunca — el almacén es estático.

`ocupada` es `public` (no `readonly`) porque cambia cuando un robot entra o sale. Lo mismo con `reservada`. Estos campos son el mecanismo de **anticolisión** y **reserva de destino** respectivamente.

`reservada` es `any | null` deliberadamente. En la primera entrega del TP el modelo definía este campo en la celda misma. Ponerle tipo estricto `Robot | null` crearía una dependencia circular entre `Celda` y `Robot`. Usar `any` es un compromiso pragmático que respeta el diseño original sin introducir acoplamiento.

```

export class Estanteria extends Celda {
  public paquetes: Paquete[] = [];

  public depositar(paquete: Paquete): void {
    if (!this.estaVacía())
      throw new Error(`Estanteria (${this.x},${this.y}) ya
tiene un paquete`);
    this.paquetes.push(paquete);
    paquete.posicion = this;
  }

  public retirar(): Paquete | null { ... }
}

```

`depositar()` tiene validación defensiva aunque `ControladorAlmacen` ya garantiza que solo asigna estanterías vacías. La razón: si en el futuro alguien llama `depositar()` directamente (desde un test, desde otro controlador), la estantería

se protege sola. Esta redundancia intencional es **programación defensiva** — el costo es mínimo y el beneficio es atrapar bugs difíciles de rastrear.

Cuando se deposita, actualizamos `paquete.posicion = this`. Esto mantiene la consistencia bidireccional: la estantería sabe qué paquete tiene, y el paquete sabe dónde está. Sigue **Information Expert** en ambas direcciones.

```
export class Muelle extends Celda {
  public acoplado: boolean = false;
  constructor(x: number, y: number, public readonly id: string) { ... }
}

export class BaseCarga extends Celda {
  constructor(x: number, y: number, public readonly id: string) { ... }
}
```

`Muelle` agrega `id` (string) y `acoplado` (boolean). El `id` permite buscarlo por nombre desde el CSV (`getMuelle('M1')`). `acoplado` indica si hay un camión amarrado — los robots no pueden pasar por un muelle acoplado.

`BaseCarga` solo agrega `id` para identificación. No tiene lógica propia porque la recarga es responsabilidad de los estados del robot, no de la celda.

SOLID:

- **SRP:** Cada subclase tiene exactamente los campos y métodos que corresponden a su tipo de celda.
- **OCP:** Agregar un nuevo tipo de celda (por ejemplo `ZonaEspera`) no requiere modificar `Celda` ni las subclases existentes.
- **LSP:** Cualquier subclase puede usarse donde se espera una `Celda`.

GRASP:

- **Information Expert:** `Estanteria` es la experta en "¿qué paquetes tengo?" y "¿estoy vacía?". Esa información no vive en el Controlador.
- **Polymorphism:** Distintos comportamientos por tipo de celda sin chequeos de tipo en el código consumidor.
- **Protected Variations:** El resto del sistema no necesita saber si una celda es estantería o pasillo para operar sobre ella como `Celda`.

Almacen.ts

```
export class Almacen {
  public readonly width: number;
  public readonly height: number;
  private celdas: Map<string, Celda> = new Map();

  constructor(width: number, height: number) {
    this.width = width;
    this.height = height;
  }
}
```

`width` y `height` son `readonly` porque las dimensiones del almacén no cambian durante la simulación. Son informativas y también se usan para validación defensiva.

`celdas` es `private` porque nadie fuera de `Almacen` debería manipular el mapa directamente — toda interacción pasa por los métodos públicos. Esto es **encapsulamiento**: el detalle de implementación (que usamos un `Map`) está oculto. Si en el futuro cambiáramos a un array 2D, el resto del sistema no se enteraría.

Usamos `Map<string, Celda>` con clave `"x,y"` en vez de un array 2D por dos razones: primero, permite grillas dispersas donde no todas las posiciones tienen celda definida; segundo, el acceso es $O(1)$ por coordenada sin necesidad de calcular índices manuales. La clave string `"x,y"` es simple y legible.

```
public agregarCelda(celda: Celda): void {
  if (celda.x < 0 || celda.x >= this.width || celda.y < 0
  || celda.y >= this.height)
    throw new Error(`Celda (${celda.x},${celda.y}) fuera
de los límites del almacén`);
  this.celdas.set(`${celda.x},${celda.y}`, celda);
}
```

La validación de límites es **programación defensiva**. `ControladorAlmacen` ya genera las celdas correctamente, pero si alguien comete un error en el CSV o en el código de inicialización, el error aparece acá con un mensaje claro en vez de silenciosamente en una posición inválida. El costo de este chequeo es negligible.

```
public getCelda(x: number, y: number): Celda | undefined
{ ... }
public getEstanterias(): Estanteria[] { ... }
public getEstanteriasVacias(): Estanteria[] { ... }
public getMuelles(): Muelle[] { ... }
public getMuelle(id: string): Muelle | undefined { ... }
public getBasesCarga(): BaseCarga[] { ... }
```

Cada método de consulta usa **TypeScript type predicates** (`c is Estanteria`) para filtrar el mapa y devolver arrays tipados. Esto permite que el consumidor reciba directamente `Estanteria[]` en vez de `Celda[]`, evitando casteos en el código que llama.

`getMuelle(id: string)` recibe un string porque los muelles se identifican por nombre en el CSV (`M1`, `M2`). No por coordenada, porque el CSV de camiones referencia el muelle por nombre, no por posición.

`getEstanteriasVacias()` existe como método propio en vez de que el Controlador filtre — siguiendo **Information Expert**: el Almacén es el experto en el estado de sus celdas.

SOLID:

- **SRP**: Almacen solo gestiona la grilla de celdas. No sabe de robots, órdenes ni simulación.
- **OCP**: Agregar `getBasesCargaLibres()` no requiere modificar la estructura existente.
- **DIP**: Depende de la abstracción `Celda`, no de tipos concretos (aunque los métodos de consulta usan `instanceof` internamente para filtrar).

GRASP:

- **Information Expert**: Almacen es el experto en "¿qué hay en (x,y)?", "¿cuáles estanterías están vacías?", "¿cuál es el muelle M1?". Esa información le pertenece a él porque es quien tiene el mapa.

- **Pure Fabrication:** Almacén no representa una entidad del mundo real en sentido estricto — es una abstracción técnica que agrupa celdas para que el resto del sistema pueda consultarlas.
 - **Low Coupling:** No conoce robots, camiones ni órdenes.
 - **High Cohesion:** Todos sus métodos tienen que ver con acceso y consulta de celdas.
-

Tarea.ts

```
export type TipoTarea = 'BUSCAR' | 'DEPOSITAR';

export class Tarea {
  constructor(
    public readonly id: string,
    public readonly tipo: TipoTarea,
    public readonly orden: Orden,
    public readonly origen: Celda,
    public readonly destino: Celda
  ) {}
}
```

`Tarea` es la clase más simple del sistema — todos sus campos son `readonly` porque una tarea es **inmutable por diseño**. Una vez que el Controlador crea una tarea y la asigna a un robot, no tiene sentido cambiar su destino ni su tipo. Si algo cambia, se crea una tarea nueva.

No hay métodos porque `Tarea` es un **Value Object** — su propósito es transportar información entre el `ControladorAlmacen` (que crea las tareas) y el `Robot` (que las ejecuta). Agregar lógica acá violaría SRP: la tarea no debería saber cómo ejecutarse.

`origen` y `destino` son referencias a objetos `Celda` del almacén, no coordenadas sueltas. Esto permite que `EstadoOperando` compare `robot.posicion === tarea.destino` por referencia de objeto directamente, sin buscar en el mapa.

`TipoTarea` como type literal en vez de enum, por la misma razón que `EstadoOrden` — liviano y suficiente.

SOLID:

- **SRP:** Tarea es solo un contenedor de datos de una asignación. No ejecuta, no decide, no valida.
- **ISP:** Interfaz mínima — ningún consumidor de Tarea necesita más que lo que expone.

GRASP:

- **Low Coupling:** No tiene lógica propia, no llama a nadie. Solo es leída.
- **Information Expert:** Sabe su propio destino y tipo. Quien necesita saber "¿a dónde va este robot?" pregunta a su tarea.

Camion.ts

```
export type TipoCamion = 'RECEPCION' | 'DESPACHO';
export type EstadoCamion = 'ACOPLADO' | 'OPERATIVO' | 'LISTO' | 'RETIRADO';

export class Camion {
  public estado: EstadoCamion = 'ACOPLADO';
  public ordenes: Orden[] = [];

  constructor(
    public readonly id: string,
    public readonly tipo: TipoCamion,
    public readonly muelle: Muelle,
    public readonly tickLlegada: number
  ) {}
}
```

El estado inicial es `'ACOPLADO'` porque cuando `ControladorAlmacen` crea el camión (dentro de `onCamionLlega()`), ya llegó físicamente. El estado `PROGRAMADO` que existe en el diagrama de la primera entrega lo maneja `SimuladorEntorno` con su lista `camionesPendientes` — mientras el camión no llegó, existe solo como DTO, no como objeto de dominio. Esta decisión evita que `Camion` tenga un estado que nunca usaría en su ciclo de vida real.

`id`, `tipo`, `muelle` y `tickLlegada` son `readonly` por la misma razón que en `Orden` — son la identidad del camión y no cambian.

```
public get x(): number { return this.muelle.x; }  
public get y(): number { return this.muelle.y; }
```

La posición del camión se deriva del muelle donde está acoplado. No guardamos `x` e `y` propios porque sería duplicar información que ya existe en `muelle` — si el muelle tiene coordenadas, el camión las hereda. Esto sigue **DRY** y evita que las coordenadas del camión y su muelle puedan desincronizarse.

```
public todasCompletadas(): boolean {  
    return this.ordenes.length > 0 && this.ordenes.every(o  
=> o.estaCompletada());  
}  
  
public ordenesActivas(): Orden[] {  
    return this.ordenes.filter(o => !o.estaCompletada());  
}
```

`todasCompletadas()` requiere que haya al menos una orden (`length > 0`) para evitar que un camión vacío se considere listo — un camión sin órdenes es un error de configuración, no un camión completo.

Ambos métodos son `public` porque los llama `ControladorAlmacen`. Son métodos del Camión y no del Controlador porque el Camión es el **Information Expert** sobre sus propias órdenes — él tiene la lista, él sabe si están todas completas.

SOLID:

- **SRP:** Camion gestiona su estado y sus órdenes. La decisión de cuándo retirarlo la toma el Controlador, no el Camión.
- **DIP:** Depende de `Orden` (abstracción del dominio) y `Muelle` (también dominio).

GRASP:

- **Information Expert:** Camion sabe si todas sus órdenes están completas. No delegamos esa consulta al Controlador.
- **Low Coupling:** No conoce robots ni el algoritmo de asignación.

- **Creator:** El Camión agrega sus propias órdenes con `agregarOrden()` — es el propietario natural de esa colección.

Robot.ts

`Robot` es la entidad más compleja del sistema porque integra tres patrones: **State** (delega comportamiento al estado actual), **Strategy** (delega navegación al algoritmo actual) y lógica de batería propia.

```
export class Robot {  
  public estado: EstadoRobot;  
  public carga: Paquete | null = null;  
  public tarea: Tarea | null = null;  
  public ultimaTarea: Tarea | null = null;  
  public ticksBloqueado: number = 0;  
  private estrategia: IEstrategiaNavegacion;
```

`estado` es `public` porque los estados necesitan asignarse desde fuera (desde el Controlador al crear robots, y desde los propios estados al hacer transiciones).

`estrategia` es `private` porque es un detalle de implementación — nadie fuera de Robot necesita saber qué algoritmo de navegación está usando.

`carga` y `tarea` son `public` porque el Controlador los lee y el robot los modifica durante `realizarOperacion()`. `ultimaTarea` es el mecanismo que usamos para que el Controlador sepa cuándo crear la segunda tarea (DEPOSITAR) — el robot guarda la última tarea completada para que en el paso 3b del tick se pueda consultar.

```
  constructor(  
    public readonly id: string,  
    public posicion: Celda,  
    public bateria: number = 100  
  ) {  
    this.estado = new EstadoInactivo();  
    this.estrategia = new MovimientoEnL();  
  }
```

`posicion` no es `readonly` porque el robot se mueve. `bateria` tampoco porque se consume y se recarga. El estado inicial siempre es `EstadoInactivo` y la estrategia inicial siempre es `MovimientoEnL` — son los valores por defecto del dominio.

```
public moverHacia(destino: Celda, almacen: Almacen): void
{
    const siguiente = this.estrategia.siguientePaso(this.posicion, destino, almacen);

    if (!siguiente) {
        this.ticksBloqueado++;
        if (this.ticksBloqueado >= 3) {
            this.estrategia = new AEstrella();
            this.ticksBloqueado = 0;
        }
        return;
    }

    this.ticksBloqueado = 0;
    this.estrategia = new MovimientoEnL();
    this.posicion.ocupada = false;
    siguiente.ocupada = true;
    this.posicion = siguiente;
    this.bateria -= this.carga ? 2 : 1;

    if (this.bateria <= 0) {
        this.bateria = 0;
        const { EstadoRecargando } = require('../states/EstadoRecargando');
        this.estado = this.posicion instanceof BaseCarga
            ? new EstadoRecargando()
            : new EstadoInmovilizado();
    }
}
```

El escalado de estrategia ocurre acá: si la estrategia devuelve `null` (bloqueado) tres veces seguidas, cambiamos a A*. Cuando se puede mover de

nuevo, volvemos a `MovimientoEnL`. Esto implementa el **patrón Strategy en runtime** — el objeto que implementa la estrategia cambia dinámicamente.

El consumo de batería es `2` con carga y `1` sin carga — el robot gasta más energía transportando peso. Cuando la batería llega a 0, si el robot cayó en una `BaseCarga` transiciona a `RECARGANDO` en vez de `INMOVILIZADO`. Este chequeo evita que el robot quede atrapado justo en el paso final hacia la base.

El `require()` dinámico para `EstadoRecargando` existe para romper la **dependencia circular**: `Robot` importa estados → los estados importan `Robot`. TypeScript en tiempo de compilación lo permite con `import type`, pero en runtime necesitamos el require diferido.

```
public necesitaRecargar(almacen: Almacen): boolean {
    const base = this.baseMasCercana(almacen);
    if (!base) return false;
    const distancia = distanciaManhattan(this.posicion.x, this.posicion.y, base.x, base.y);
    const costo = this.carga ? 2 : 1;
    return this.bateria <= distancia * costo;
}
```

Esta es la **gestión proactiva de batería** que pide la consigna. El robot calcula si le alcanza la batería para llegar a la base más cercana considerando el costo de movimiento (con o sin carga). Si no le alcanza, entra en `BATERIA_BAJA` antes de quedarse sin energía. Usa **distancia Manhattan** porque en una grilla solo se mueve horizontal y verticalmente — la distancia euclidiana subestimaría el camino real.

SOLID:

- **SRP**: Robot gestiona posición, batería, carga y tarea. La lógica de comportamiento por estado vive en `EstadoRobot`. La lógica de pathfinding vive en `IEstrategiaNavegacion`.
- **OCP**: Agregar un nuevo estado o estrategia no requiere modificar Robot.
- **DIP**: Depende de `EstadoRobot` e `IEstrategiaNavegacion` (interfaces/abstracciones), no de clases concretas.

GRASP:

- **Information Expert:** Robot es el experto en su propia posición, batería y carga.
- **Polymorphism:** El comportamiento por tick varía según el estado actual — sin `if` ni `switch`.
- **Protected Variations:** El algoritmo de navegación está oculto detrás de `IEstrategiaNavegacion`. Cambiar de A* a Dijkstra no afectaría a Robot.
- **Low Coupling:** Robot no conoce otros robots, camiones ni el Controlador.

EstadoRobot.ts (interfaz)

```
export interface EstadoRobot {
  readonly nombre: string;
  ejecutar(robot: Robot, almacen: Almacen): void;
}
```

Esta interfaz define el contrato del **patrón State**. Es deliberadamente mínima: solo dos miembros. `nombre` es `readonly` porque es una constante de identidad del estado — sirve para logging y para que el frontend muestre en qué estado está el robot. `ejecutar()` es el único comportamiento que un estado necesita exponer: qué hace el robot en este tick.

Pasamos tanto `robot` como `almacen` porque los estados necesitan los dos: `robot` para leer y modificar su posición, batería y estado; `almacen` para consultar celdas vecinas, bases de carga y calcular distancias. Si solo pasáramos `robot`, los estados no podrían navegar. Si solo pasáramos `almacen`, no podrían modificar al robot.

La interfaz está en `domain/interfaces/` y no en `domain/states/` porque es una abstracción que trasciende las implementaciones — el `Robot` depende de ella, y `Robot` no debería depender de la carpeta de implementaciones.

SOLID:

- **ISP:** La interfaz tiene exactamente lo necesario. Ningún estado está obligado a implementar métodos que no usa.
- **DIP:** `Robot` depende de `EstadoRobot` (abstracción), no de `EstadoInactivo` ni `EstadoOperando` (concretos).

GRASP:

- **Polymorphism:** Define el punto de variación — cada estado implementa `ejecutar()` de forma distinta.
- **Protected Variations:** Oculta las diferencias de comportamiento entre estados detrás de una interfaz estable.

IEstrategiaNavegacion.ts (interfaz)

```
export interface IEstrategiaNavegacion {  
    siguientePaso(origen: Celda, destino: Celda, almacen: Almacen): Celda | null;  
}
```

Contrato del **patrón Strategy** para navegación. Un solo método: dado dónde estás, adónde querés ir y cómo es el mapa, devolvé la celda a la que deberías moverte ahora.

Devuelve `Celda | null` y no una lista de celdas porque el robot se mueve **un paso por tick**. Devolver el camino completo acoplaría la estrategia con el mecanismo de ejecución del robot. Con un solo paso, el robot puede interrumpir el plan en cualquier momento (por ejemplo, si detecta que necesita recargar).

El `null` indica "bloqueado" — no hay paso posible en este tick. Es el robot quien decide qué hacer ante un bloqueo (incrementar `ticksBloqueado` y eventualmente escalar a A*).

SOLID:

- **ISP:** Un solo método. Ninguna implementación carga con métodos innecesarios.
- **DIP:** `Robot` depende de esta interfaz, no de `MovimientoEnL` ni `AEstrella`.

GRASP:

- **Polymorphism:** Cada algoritmo implementa `siguientePaso()` de forma distinta.
- **Protected Variations:** `Robot` no sabe si está usando L-movement o A* — eso cambia en runtime sin que Robot lo note.

EstadoInactivo.ts

```
export class EstadoInactivo implements EstadoRobot {
    public readonly nombre = 'INACTIVO';

    ejecutar(robot: Robot, almacen: Almacen): void {
        const celda = robot.posicion;

        if (celda instanceof BaseCarga && robot.bateria < 100)
        {
            robot.estado = new EstadoRecargando();
            return;
        }

        if (celda instanceof Muelle || celda instanceof Estanteria) {
            robot.moverHaciaPasilloMasCercano(almacen);
        }
    }
}
```

Un robot INACTIVO no tiene tarea asignada. Su comportamiento en cada tick es simple: si está en una celda especial que no debería ocupar, se mueve.

El primer chequeo es `BaseCarga`: si el robot está inactivo sobre una base de carga y no está a full, aprovecha para recargar. Esto cubre el caso donde un robot termina su tarea con la batería baja y casualmente está en una base — transiciona a `RECARGANDO` sin necesidad de pasar por `BATERIA_BAJA`.

El segundo chequeo es `Muelle` o `Estanteria`: estas celdas son funcionales — las ocupa un camión o un paquete. Un robot inactivo parado ahí bloquea las operaciones. `moverHaciaPasilloMasCercano()` lo desplaza a una celda libre adyacente.

Si el robot está en un pasillo, no hace nada. Espera ahí hasta que el Controlador le asigne una tarea.

SOLID:

- **SRP:** Solo define qué hace un robot inactivo. No sabe de órdenes ni de camiones.
- **OCP:** Si hubiera un nuevo tipo de celda especial, se agrega un `instanceof` acá sin tocar otros estados.

GRASP:

- **Polymorphism:** Comportamiento completamente distinto al de `EstadoOperando` o `EstadoRecargando`, expresado como clase separada y no como `if (estado === 'INACTIVO')`.
- **Low Coupling:** Solo conoce `Robot`, `Almacen`, `BaseCarga`, `Muelle` y `Estanteria` — todos del dominio.

EstadoOperando.ts

```
export class EstadoOperando implements EstadoRobot {
  public readonly nombre = 'OPERANDO';

  ejecutar(robot: Robot, almacen: Almacen): void {
    if (!robot.tarea) return;

    if (robot.necesitaRecargar(almacen)) {
      robot.estado = new EstadoBateriaBaja();
      robot.estado.ejecutar(robot, almacen);
      return;
    }

    if (robot.bateria <= 0) {
      robot.estado = new EstadoInmovilizado();
      return;
    }

    const destino = robot.tarea.destino;

    if (robot.posicion === destino) {
      robot.realizarOperacion();
      if (!robot.tarea) {
```

```

        robot.estado = new EstadoInactivo();
    }
} else {
    robot.moverHacia(destino, almacen);
}
}
}
}

```

Este es el estado más complejo porque concentra la lógica de ejecución de tareas. El orden de los chequeos es deliberado y tiene consecuencias:

Primero se chequea si hay tarea. Si no hay, el método retorna sin hacer nada — esto es una guardia defensiva, porque `EstadoOperando` sin tarea es un estado inconsistente que puede ocurrir transitoriamente entre ticks.

Segundo se chequea `necesitaRecargar()` antes de cualquier movimiento. Si el robot no tiene batería suficiente para llegar a la base, transiciona a `BATERIA_BAJA` inmediatamente y llama `ejecutar()` en el mismo tick — esto hace que el robot empiece a moverse hacia la base en el mismo tick que detecta el problema, sin desperdiciar un tick.

Tercero se chequea `bateria <= 0` como red de seguridad. En teoría `necesitaRecargar()` lo previene, pero si por alguna razón el robot llega a batería cero estando OPERANDO, transiciona a `INMOVILIZADO`.

Cuarto se chequea si el robot ya llegó al destino: si sí, ejecuta la operación. Si `realizarOperacion()` limpia la tarea (la pone en `null`), el robot transiciona a `INACTIVO`. Si no la limpia (porque la operación no se pudo completar — por ejemplo no había carga), la tarea sigue activa.

Quinto si no llegó, se mueve un paso hacia el destino con `moverHacia()`.

SOLID:

- **SRP:** Solo define el comportamiento del robot cuando está operando. No sabe de asignación de órdenes ni de cómo se crea la siguiente tarea.
- **OCP:** Agregar un nuevo chequeo (por ejemplo colisión con otro robot) no requiere modificar otros estados.

GRASP:

- **Polymorphism:** Implementa `ejecutar()` de forma completamente distinta a los otros estados.

- **Information Expert:** Consulta `robot.necesitaRecargar()` y `robot.tarea.destino` — delega el conocimiento a quien lo tiene.

EstadoBateriaBaja.ts

```
export class EstadoBateriaBaja implements EstadoRobot {
  public readonly nombre = 'BATERIA_BAJA';

  ejecutar(robot: Robot, almacen: Almacen): void {
    console.log(`[BATERIA_BAJA ${robot.id}] ...`);

    if (robot.posicion instanceof BaseCarga) {
      robot.estado = new EstadoRecargando();
      return;
    }

    const base = almacen.getBasesCarga()
      .filter(b => !b.ocupada)
      .sort((a, b) => {
        const da = Math.abs(robot.posicion.x - a.x) + Math.
abs(robot.posicion.y - a.y);
        const db = Math.abs(robot.posicion.x - b.x) + Math.
abs(robot.posicion.y - b.y);
        return da - db;
      })[0];

    if (!base) return;
    robot.moverHacia(base, almacen);
  }
}
```

El primer chequeo es `instanceof BaseCarga` — si el robot ya está sobre una base, transiciona a `RECARGANDO` directamente. Esto resuelve el bug donde `baseMasCercana()` podría devolver una base distinta a la que el robot ya ocupa (porque recalcula distancias cada tick), haciendo que el robot nunca transicione a `RECARGANDO`.

La búsqueda de base filtra por `!b.ocupada` antes de ordenar. Esto evita que dos robots con batería baja apunten a la misma base — el segundo robot ve esa base como ocupada y elige la siguiente más cercana. Sin este filtro, ambos robots intentarían llegar al mismo punto y uno terminaría bloqueado indefinidamente.

Ordenamos por distancia Manhattan inline en vez de llamar a `baseMasCercana()` porque `baseMasCercana()` no filtra por ocupación. Duplicar el cálculo de distancia es un compromiso aceptable para tener el filtro.

Si no hay ninguna base libre (`!base`), el método retorna sin hacer nada. El robot queda en `BATERIA_BAJA` hasta que alguna base se libere.

SOLID:

- **SRP:** Solo define cómo reacciona un robot con batería baja.

GRASP:

- **Information Expert:** Consulta `almacen.getBasesCarga()` — el Almacén es quien sabe dónde están las bases.
- **Protected Variations:** El mecanismo de selección de base puede cambiar sin afectar a otros estados.

EstadoRecargando.ts

```
export class EstadoRecargando implements EstadoRobot {
    public readonly nombre = 'RECARGANDO';

    ejecutar(robot: Robot, _almacen: Almacen): void {
        robot.bateria = Math.min(100, robot.bateria + 10);

        if (robot.bateria === 100) {
            robot.estado = robot.tarea
                ? new EstadoOperando()
                : new EstadoInactivo();
        }
    }
}
```

El estado más simple de los cinco. Cada tick suma 10 de batería con un tope de 100 — `Math.min` evita que la batería supere el máximo sin necesidad de un `if`.

Cuando llega a 100, la transición depende de si el robot tiene tarea pendiente. Si tenía una tarea asignada antes de ir a recargar (por ejemplo, estaba OPERANDO, detectó batería baja, fue a recargar pero la tarea sigue ahí), vuelve a `OPERANDO`. Si no tiene tarea, va a `INACTIVO`.

`_almacen` tiene el prefijo `_` para indicar explícitamente que el parámetro existe (porque la firma de la interfaz lo requiere) pero no se usa en este estado. Es una convención TypeScript que evita warnings del compilador y comunica intención al lector.

SOLID:

- **SRP:** Solo carga batería y decide la transición al completar.

GRASP:

- **Low Coupling:** No necesita consultar el almacén, no necesita conocer otros robots.
- **Information Expert:** Robot sabe su propia batería y su propia tarea — `EstadoRecargando` los consulta directamente.

EstadoInmovilizado.ts

```
export class EstadoInmovilizado implements EstadoRobot {
  public readonly nombre = 'INMOVILIZADO';

  ejecutar(robot: Robot, _almacen: Almacen): void {
    if (robot.posicion instanceof BaseCarga) {
      robot.estado = new EstadoRecargando();
    }
  }
}
```

El estado terminal del robot — batería en cero, no puede moverse. La única salida es si el robot está parado sobre una base de carga, en cuyo caso transiciona a `RECARGANDO`.

Este chequeo cubre el caso donde el robot gasta el último punto de batería exactamente en el paso que lo lleva a la base. Sin este chequeo, el robot quedaría **INMOVILIZADO** para siempre a centímetros de salvarse.

Si no está en una base, `ejecutar()` no hace nada. El robot queda bloqueado indefinidamente — esto es una **limitación conocida del sistema**, no un bug. La consigna define **INMOVILIZADO** como estado terminal sin mecanismo de rescate externo.

SOLID:

- **SRP:** Solo maneja el comportamiento de un robot sin batería.

GRASP:

- **Polymorphism:** Implementa `ejecutar()` con lógica mínima — diferente de todos los otros estados.

MovimientoEnL.ts

```
export class MovimientoEnL implements IEstrategiaNavegacion
{
    siguientePaso(origen: Celda, destino: Celda, almacen: Almacén): Celda | null {
        const dx = destino.x - origen.x;
        const dy = destino.y - origen.y;

        if (dx !== 0) {
            const siguiente = almacen.getCelda(origen.x + Math.sign(dx), origen.y);
            if (siguiente && !siguiente.ocupada) return siguiente;
        }

        if (dy !== 0) {
            const siguiente = almacen.getCelda(origen.x, origen.y + Math.sign(dy));
            if (siguiente && !siguiente.ocupada) return siguiente;
        }
    }
}
```

```

    }

    return null;
  }
}

```

La estrategia por defecto — simple, rápida, $O(1)$. Prioriza movimiento horizontal: si hay diferencia en X, intenta moverse en X primero. Solo intenta Y si X está bloqueado o ya está alineado.

`Math.sign(dx)` devuelve -1, 0 o 1 — el paso en la dirección correcta sin calcular distancias. Esto garantiza que el robot siempre se acerque al destino, nunca se aleje.

Devuelve `null` si ambas direcciones están bloqueadas. El robot acumula ticks bloqueado hasta que escala a A*. La prioridad X-primero es arbitraria pero **determinista** — siempre toma la misma decisión ante la misma situación, lo que hace la simulación reproducible.

SOLID:

- **SRP:** Solo implementa L-movement. No decide cuándo usarse ni cuándo escalar.
- **DIP:** Implementa `IEstrategiaNavegacion`.

GRASP:

- **Polymorphism:** Algoritmo intercambiable con A* sin que Robot lo note.
- **Low Coupling:** Solo conoce `Celda` y `Almacen` — las abstracciones mínimas necesarias.

AEstrella.ts

```

export class AEstrella implements IEstrategiaNavegacion {
  siguientePaso(origen: Celda, destino: Celda, almacen: Almacen): Celda | null {
    // open set ordenado por f = g + h
    // closed set como Set<string> para O(1)
    // reconstrucción del camino por cadena de padres
  }
}

```



```
// devuelve el primer paso del camino óptimo
}
}
```

Implementación del algoritmo A* con heurística Manhattan. Se usa como fallback cuando `MovimientoEnL` falla tres ticks seguidos — garantiza encontrar el camino si existe uno.

La **heurística Manhattan** es admisible en una grilla sin movimiento diagonal: nunca sobreestima el costo real. Esto garantiza que A* encuentra el camino óptimo.

El **open set** se implementa como array ordenado por `f = g + h` (costo acumulado + heurística). Para los tamaños de grilla típicos del TP (10×10), el overhead de ordenar el array es negligible. Una priority queue sería más eficiente en grillas grandes pero agrega complejidad innecesaria.

El **closed set** es `Set<string>` con claves `"x,y"` — O(1) para consulta y inserción. Evita visitar nodos ya procesados.

La reconstrucción del camino sigue la cadena de padres desde el destino hasta el origen y devuelve **solo el primer paso** — consistente con la interfaz `IEstrategiaNavegacion` que devuelve `Celda | null`. Esto mantiene la compatibilidad con `MovimientoEnL` y permite que el robot interrumpa el plan en cualquier tick.

SOLID:

- **SRP:** Solo implementa A*. No decide cuándo activarse.
- **DIP:** Implementa `IEstrategiaNavegacion`.

GRASP:

- **Polymorphism:** Intercambiable con `MovimientoEnL`.
- **Protected Variations:** El robot no sabe que cambió de algoritmo — la interfaz es la misma.

distanciaManhattan.ts

```
export function distanciaManhattan(
  ax: number, ay: number,
  bx: number, by: number
```

```

): number {
  return Math.abs(ax - bx) + Math.abs(ay - by);
}

```

Función pura — sin estado, sin efectos secundarios, determinista. Se usa en tres lugares: `AEstrella` (heurística), `EstadoBateriaBaja` (ordenar bases por distancia) y `Robot.necesitaRecargar()` (calcular si alcanza la batería).

La extrajimos porque se repetía tres veces. Una función duplicada en tres lugares es una fuente de bugs: si hay que cambiar el cálculo (por ejemplo, agregar peso a las diagonales), se tiene que cambiar en tres lugares. Extraerla sigue **DRY** y **SRP**.

Usamos Manhattan y no Euclidea porque en una grilla donde solo se mueve horizontal y verticalmente, la distancia Euclidea subestimaría el camino real — un robot a diagonal no puede llegar en `sqrt(2)` pasos, necesita 2.

SOLID:

- **SRP:** Hace exactamente una cosa.

GRASP:

- **Information Expert:** Encapsula el conocimiento de cómo medir distancias en este dominio.
- **Pure Fabrication:** No representa ninguna entidad del mundo real — es una abstracción técnica al servicio del dominio.

ControladorAlmacen.ts

```

export class ControladorAlmacen {
  private almacen!: Almacen;
  private robots: Robot[] = [];
  private camiones: Camion[] = [];
  private ordenesLibres: Orden[] = [];
  private contadorTareas: number = 0;
}

```

Todos los campos son `private` porque son estado interno del controlador — ningún módulo externo debería leer ni modificar directamente la lista de robots o camiones. El acceso es siempre a través de métodos públicos.

`almacen` tiene `!` (non-null assertion) porque TypeScript no puede saber que `inicializar()` se llama antes de cualquier otro método. Es una limitación del lenguaje, no del diseño.

`contadorTareas` genera IDs únicos para tareas (`T1`, `T2`...) sin depender de ningún generador externo. Simple y suficiente para el scope del TP.

```
public inicializar(mapaConfig: MapaConfigDTO, robotsConfig: RobotConfigDTO[]): void {
    this.robots = [];
    this.camiones = [];
    this.ordenesLibres = [];
    this.contadorTareas = 0;
    // ... crear almacén, celdas y robots
}
```

Lo primero que hace `inicializar()` es limpiar todos los arrays. Esto permite que el método se llame múltiples veces (reinicio de simulación) sin acumular estado de la sesión anterior. Sin esta limpieza, al reiniciar los robots nuevos se sumaban a los viejos — fue uno de los bugs que corregimos.

La inicialización del mapa sigue dos pasos: primero pobla todo como `Pasillo`, después sobrescribe con los tipos especiales (estanterías, muelles, bases). Esto simplifica el CSV — solo hace falta listar las celdas especiales, no todas las celdas del mapa.

```
public procesarPaso(): void {
    // 1. Liberar órdenes de camiones ACOPLADO → OPERATIVO
    // 2. Asignar órdenes a robots INACTIVOS
    // 3. Cada robot ejecuta su acción
    // 3b. Crear tarea DEPOSITAR para robots que completaron BUSCAR con carga
    // 4. Cada camión evalúa si puede retirarse
}
```

El orden del tick es deliberado y tiene consecuencias:

Paso 1 antes que **Paso 2**: las órdenes deben existir antes de poder asignarse. Si invertimos el orden, el tick en que llega el camión no se asignarían órdenes.

Paso 2 antes que **Paso 3**: los robots reciben su tarea antes de ejecutar. Si el orden fuera al revés, el robot ejecutaría INACTIVO un tick antes de recibir su tarea.

Paso 3b después de **Paso 3**: el robot debe completar su tarea BUSCAR (en el paso 3) para que en el paso 3b se detecte que tiene carga y se cree la tarea DEPOSITAR. No pueden ir en el mismo paso.

Paso 4 al final: el camión no debería retirarse hasta que todos los robots hayan ejecutado su acción en ese tick.

```
private asignarOrdenes(): void {
    const ordenesOrdenadas = [...this.ordenesLibres].sort
((a, b) => {
    if (a.Comestible && !b.Comestible) return -1; // COMES
TIBLE primero
    if (a.Comestible && b.Comestible)
        return a.vencimiento - b.vencimiento; // más
próximo a vencer primero
    return b.peso - a.peso; // más
pesado primero
    });

    for (const robot of this.robots) {
        if (!(robot.estado instanceof EstadoInactivo)) contin
ue;
        if (celda instanceof Muelle || celda instanceof Estan
teria) continue;
        // ...
        ordenesOrdenadas.splice(ordenesOrdenadas.indexOf(orde
n), 1); // ← bug fix clave
    }
}
```

El ordenamiento implementa **FEFO**: comestibles primero (riesgo de vencimiento), dentro de comestibles los más próximos a vencer, luego generales por peso descendente.

Solo asignamos a robots **INACTIVO** — un robot operando, recargando o inmovilizado no puede recibir una nueva tarea. Además saltamos robots

parados en muelle o estantería — están en una posición que bloquea operaciones, deben moverse al pasillo primero.

El `splice` después de asignar es el fix del bug donde dos robots recibían la misma orden. `ordenesOrdenadas` es una copia del array original — sin el `splice`, el segundo robot encontraba la misma orden en la copia aunque ya hubiera sido asignada.

```
private crearTareaDepositar(robot: Robot, orden: Orden):  
void {  
    if (camion.tipo === 'RECEPCION') {  
        // reserva estantería vacía → destino = estantería  
    } else {  
        // reserva muelle del camión → destino = muelle  
    }  
    robot.estado = new EstadoOperando();  
}
```

La segunda tarea se crea acá, no en `asignarOrdenARobot()`, porque en el momento de asignar la primera tarea todavía no sabemos en qué estantería va a terminar el paquete — para RECEPCION, la estantería se elige en el momento en que el robot ya tiene el paquete y está listo para ir a depositarlo.

Después de crear la tarea DEPOSITAR, forzamos `robot.estado = new EstadoOperando()`. Esto es necesario porque el robot acaba de completar BUSCAR y transitó a INACTIVO — necesita volver a OPERANDO para ejecutar la nueva tarea en el próximo tick.

SOLID:

- **SRP:** Orquesta el tick. No ejecuta lógica de dominio — la delega a robots, estados y camiones.
- **DIP:** Depende de abstracciones del dominio (`Robot`, `Camion`, `Orden`, `Tarea`), no de infraestructura.

GRASP:

- **Controller:** Es el caso de uso principal del sistema — el punto de entrada para todo lo que pasa en un tick.
- **Pure Fabrication:** No representa ninguna entidad del mundo real. Es una clase técnica que coordina las entidades del dominio.

- **Creator:** Crea `Tarea` porque tiene toda la información necesaria (robot, orden, camión, estantería disponible).
- **Low Coupling:** No conoce el frontend, la base de datos ni la infraestructura. Recibe DTOs y devuelve objetos planos.
- **High Cohesion:** Todos sus métodos colaboran para procesar un tick de simulación.